

Approximation of FP Multiplication with MBE Neurons

1. 개별 피연산자를 스파이크 시퀀스로 표현 (Approximate Identity Mapping)

먼저, 곱셈의 두 피연산자, 즉 x_1 과 x_2 를 각각 MBE 뉴런을 사용하여 스파이크 시퀀스로 변환한다. 이 때, MBE 뉴런은 입력값 x 를 거의 그대로 (approximate identity mapping) 스파이크 신호로 변환하는 역할을 한다.

$$x \approx \text{MBE}_{Id}(x) = \sum_{t=0}^{T-1} d[t]s[t]$$

- x 는 입력값 (float)이다.
- T 는 총 타임스텝(timestep) 수이다.
- $s[t] \in 0, 1$ 은 각 타임스텝 t 에서 생성되는 바이너리 스파이크이다.
- $d[t]$ 는 각 타임스텝 t 에서 스파이크의 강도(intensity)를 나타내는 값이다.
- $\sum_{t=0}^{T-1} d[t]s[t]$ 는 T 개의 타임스텝에 걸쳐 생성된 스파이크들의 가중치 합으로, 원래 입력값 x 를 근사한다.
즉, 원래의 연속적인 부동소수점 값 x 를 T 개의 타임스텝 동안 발생하는 스파이크들의 시퀀스와 각 스파이크의 강도로 표현하는 것이다. 이 과정 자체는 MBE 뉴런의 근사 능력을 활용한다.

2. 스파이크 시퀀스를 이용한 곱셈 근사

개별 피연산자 x_1 과 x_2 가 스파이크 시퀀스 s 와 강도 벡터 d 로 표현되었다면, 이제 이 스파이크 표현들을 이용하여 $x_1 \cdot x_2$ 를 근사한다.

$$x_1 \cdot x_2 \approx \left(\sum_{i=0}^{T-1} d[i]s[i] \right) \cdot \left(\sum_{j=0}^{T-1} d'[j]s'[j] \right)$$

이 식을 전개하면 다음과 같은 형태가 된다. 여기서 d 와 s 는 x_1 에 대한 것이고, d' 와 s' 는 x_2 에 대한 것이다.

$$x_1 \cdot x_2 \approx \sum_{i=0}^{T-1} \sum_{j=0}^{T-1} d[i]d'[j] \cdot s[i]s[j]$$
 이들을 이용하여 다음과 같은 행렬을 정의할 수 있다.

- 강도 행렬 (Intensity Matrix) $D \in \mathbb{R}^{T \times T}$:

$$D_{ij} = d[i]d'[j]$$

이는 d 와 d' 의 외적(outer product)으로 볼 수 있다. $D = d^T d'$ (여기서 d^T 는 d 의 전치 행렬이다).

- 스파이크 행렬 (Spike Matrix) $S \in \{0, 1\}^{T \times T}$:

$$S_{ij} = s[i]s'[j]$$

이는 s 와 s' 의 외적(outer product)으로 볼 수 있으며, Hadamard product $s^T s'$ 와 같다.
 $S = s^T s'$

이제 원래의 곱셈 근사식은 다음과 같이 표현된다.

$$x_1 \cdot x_2 \approx \sum_{i=0}^{T-1} \sum_{j=0}^{T-1} D_{ij} \cdot S_{ij} = (D \odot S)_{\text{sum}}$$

여기서 $D \odot S$ 는 두 행렬의 요소별 곱(Hadamard product)을 의미하며, $(D \odot S)_{\text{sum}}$ 은 그 결과 행렬의 모든 요소의 합을 나타낸다.

의의

이 방식의 가장 큰 장점은 최종적인 곱셈 연산이 스파이크 행렬 S (바이너리 값)와 미리 계산될 수 있는 강도 행렬 D (최대 $N \times N$ 개의 d 값 조합으로 구성될 수 있으며, N 은 뉴런의 총 개수)를 사용하여 수행된다는 점이다. 즉, FP 곱셈이 필요한 모든 곳에서 스파이크 기반의 계산으로 대체된다.

강도 행렬 D 는 한 번 계산되어 네트워크 전체에서 공유될 수 있다.

실제 연산은 스파이크 행렬 S (희소할 수 있음)와 D 의 요소별 곱셈 및 합산으로 이루어지므로, FP 곱셈보다 에너지 효율성이 훨씬 높다.